

Labo 04 — Labo pratique : construire l'image de l'API

Objectif : écrire vous-même le Dockerfile d'une (fausse) API Spring Boot, et provoquer chacun des pièges du cours — contexte, cache, `CMD` / `ENTRYPOINT`, forme shell, `USER`.

Prérequis — Labos 01 à 03 terminés.

Fichiers fournis (`files/`) - `Api.java` — une API HTTP de 30 lignes, sans dépendance. Elle expose `/` et `/actuator/health`, lit `APP_MESSAGE`, `APP_PROFILE`, `SERVER_PORT` dans l'environnement, et gère `SIGTERM`. **Vous n'aurez jamais à la modifier** : les labos portent sur Docker, pas sur Java. - `construire-jar.sh` — compile `Api.java` en `api.jar` dans un conteneur jetable, sans installer de JDK sur votre machine.

Étape 1 — Préparer le projet

```
mkdir -p ~/labo-docker/04 && cd ~/labo-docker/04
cp <chemin-du-labo>/files/Api.java .
cp <chemin-du-labo>/files/construire-jar.sh . && chmod +x construire-jar.sh
./construire-jar.sh
```

Observez le téléchargement de `eclipse-temurin:21-jdk` (une seule fois), puis `api.jar` d'environ 2,3 Ko.

Explication. Vous venez d'utiliser un conteneur comme **outil jetable** : la compilation a eu lieu dans un JDK complet, monté sur votre dossier, et il n'en reste rien. C'est déjà l'idée du multi-stage du labo 05.

Étape 2 — Un premier Dockerfile

Créez `Dockerfile` :

```
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY api.jar /app/api.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/app/api.jar"]
CMD ["--spring.profiles.active=prod"]
```

```
docker build -t api-labo:1.0 .
docker images --format 'table {{.Repository}}\t{{.Tag}}\t{{.Size}}' | grep api-labo
```

Observez les étapes numérotées du build, puis une image d'environ **285 Mo** — pour un JAR de 2,3 Ko. L'essentiel est le JRE.

```
docker run -d --name api -p 18080:8080 api-labo:1.0
curl -s localhost:18080/ ; echo
curl -s localhost:18080/actuator/health ; echo
docker logs api
```

Observez `{"message":"Bonjour depuis l'API","profile":"default"}`, `{"status":"UP"}`, et les lignes `Arguments recus : --spring.profiles.active=prod` puis `API demarree sur le port 8080 (profil default)`.

Explication. `EXPOSE 8080` n'a rien publié : c'est `-p 18080:8080` qui a créé la redirection. Vérifiez-le en relançant sans `-p` — `curl` échouera.

Étape 3 — Le contexte de construction

```
mkdir -p ../commun && echo "cle-privee" > ../commun/secret.txt
printf 'FROM alpine\nCOPY ../commun/secret.txt /\n' > Dockerfile.hors-contexte
docker build -f Dockerfile.hors-contexte -t essai .
```

Observez l'échec : `failed to compute cache key: ... "/secret.txt": not found`.

Explication. Le fichier n'a jamais été envoyé au daemon. Ce n'est pas un problème de droits : il est physiquement absent de l'archive du contexte.

Mesurez maintenant le coût du contexte :

```
mkdir -p node_modules && dd if=/dev/zero of=node_modules/gros.bin bs=1M count=200 2>/dev/null
docker build -t api-labo:1.0 . 2>&1 | grep -i "transferring context"
```

Observez une ligne `transferring context: 200MB` (ou plus) et un build sensiblement plus lent.

```
printf 'node_modules\n*.bin\nDockerfile.hors-contexte\n' > .dockerignore
docker build -t api-labo:1.0 . 2>&1 | grep -i "transferring context"
```

Observez un contexte retombé à quelques kilo-octets.

Explication. `.dockerignore` agit **avant** toute instruction. Sans lui, ces 200 Mo seraient transférés à chaque build — et, avec un `COPY . .`, embarqués dans l'image publiée.

Étape 4 — Le cache, et l'ordre des instructions

Ajoutez une étape « dépendances » simulée. Remplacez votre `Dockerfile` par :

```
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
RUN echo "telechargement des dependances..." && sleep 5
COPY api.jar /app/api.jar
ENTRYPOINT ["java", "-jar", "/app/api.jar"]
```

```
time docker build -t api-labo:2.0 .
time docker build -t api-labo:2.0 .
```

Observez un premier build d'environ 6 secondes, puis un second quasi instantané, avec des lignes `CACHED`.

Modifiez le JAR et reconstruisez :

```
touch Api.java && ./construire-jar.sh >/dev/null
time docker build -t api-labo:2.1 .
```

Observez que l'étape `RUN ... sleep 5` reste `CACHED` : seul le `COPY` et ce qui suit sont rejoués.

Maintenant inversez l'ordre — le `COPY` **avant** le `RUN` :

```
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY api.jar /app/api.jar
RUN echo "telechargement des dependances..." && sleep 5
ENTRYPOINT ["java", "-jar", "/app/api.jar"]
```

```
docker build -t api-labo:3.0 .
./construire-jar.sh >/dev/null && time docker build -t api-labo:3.1 .
```

Observez que les 5 secondes sont **payées à chaque fois**.

Explication. Voilà, en miniature, la différence entre un build Maven de 40 secondes et un de 6 minutes : le `COPY` volatil placé avant l'étape coûteuse invalide tout ce qui suit.

Étape 5 — `CMD` contre `ENTRYPOINT`

L'image de l'étape 2 a un `ENTRYPOINT` **et** un `CMD`. Observez la combinaison :

```
timeout 5 docker run --rm api-labo:1.0 | head -2
timeout 5 docker run --rm api-labo:1.0 --debug | head -2
```

Observez dans le premier cas Arguments recus : `--spring.profiles.active=prod`, dans le second Arguments recus : `--debug`. L'`ENTRYPOINT` (`java -jar ...`) est resté ; seul le `CMD` a été remplacé.

Comparez avec une image qui n'a qu'un `CMD` :

```
printf 'FROM eclipse-temurin:21-jre-alpine\nCOPY api.jar /app/api.jar\nCMD ["java", "-jar", "/app/api.jar"]\n' > D-cmd
docker build -q -f D-cmd -t api-labo:cmd .
docker run --rm api-labo:cmd sh -c 'echo "je remplace tout"'
```

Observez que l'API ne démarre **pas du tout** : avec un `CMD` seul, l'argument remplace la commande entière.

```
docker run --rm --entrypoint sh api-labo:1.0 -c 'echo "shell obtenu malgré ENTRYPOINT"'
```

Observez que `--entrypoint` est la porte de sortie pour déboguer.

Explication. `CMD` = valeur par défaut remplaçable, `ENTRYPOINT` = programme fixe auquel on ajoute des arguments. Le motif d'entreprise est `ENTRYPOINT` + `CMD` pour les arguments par défaut.

Étape 6 — La forme *shell*, ou l'arrêt cassé

C'est l'expérience la plus importante du labo. Construisez la **même** application avec la forme *shell*, sur une base **Debian** :

```
printf 'FROM eclipse-temurin:21-jre\nCOPY api.jar /app/api.jar\nENTRYPOINT java -jar /app/api.jar\n' > D-shell-debian
docker build -q -f D-shell-debian -t api-labo:shell-debian .
docker run -d --name s-deb api-labo:shell-debian
sleep 3
docker exec s-deb ps -o pid,args | head -3
```

Observez :

```
PID COMMAND
1 /bin/sh -c java -jar /app/api.jar
7 java -jar /app/api.jar
```

Le shell est resté PID 1. Arrêtez :

```
time docker stop s-deb
docker inspect --format 'code={{.State.ExitCode}}' s-deb
docker logs s-deb | tail -2
```

Observez `real 0m10.1s`, `code=137`, et **aucun** message « SIGTERM reçu » : les *shutdown hooks* n'ont pas été exécutés.

Comparez avec la forme *exec* de l'étape 2 :

```
docker run -d --name s-exec api-labo:1.0
sleep 3 ; docker exec s-exec ps -o pid,args | head -3
time docker stop s-exec
docker inspect --format 'code={{.State.ExitCode}}' s-exec
docker logs s-exec | tail -2
```

Observez `1 java -jar /app/api.jar`, un arrêt en **0,2 s**, `code=143`, et les lignes `SIGTERM reçu` : arrêt propre en cours... puis `API arretee proprement`.

Enfin, la même forme *shell* mais sur base **Alpine** :

```
printf 'FROM eclipse-temurin:21-jre-alpine\nCOPY api.jar /app/api.jar\nENTRYPOINT java -jar /app/api.jar\n' > D-shell-alpine
docker build -q -f D-shell-alpine -t api-labo:shell-alpine .
docker run -d --name s-alp api-labo:shell-alpine ; sleep 3
docker exec s-alp ps -o pid,args | head -3
time docker stop s-alp ; docker inspect --format 'code={{.State.ExitCode}}' s-alp
```

Observez que cette fois Java **est** PID 1 et que l'arrêt est propre (143).

Explication. Le shell de busybox (Alpine) se remplace lui-même par la commande quand celle-ci est simple ; le `dash` de Debian ne le fait pas. **Le même Dockerfile a donc deux comportements selon l'image de base.** C'est exactement le genre de bogue qui fonctionne sur le poste du développeur et casse en production. La forme `exec` supprime la question.

```
docker rm s-deb s-exec s-alp
```

Étape 7 — Le script d'entrée sans `exec`

Le cas le plus fréquent en entreprise :

```
printf '#!/bin/sh\neco "preparation..."njava -jar /app/api.jar\n' > entrypoint.sh
chmod +x entrypoint.sh
printf 'FROM eclipse-temurin:21-jre-alpine\nCOPY api.jar /app/api.jar\nCOPY entrypoint.sh /entrypoint.sh\nENTRYPOINT ["/entrypoint.sh"]\n' > D-script
docker build -q -f D-script -t api-labo:script .
docker run -d --name s-script api-labo:script ; sleep 3
docker exec s-script ps -o pid,args | head -4
time docker stop s-script
docker inspect --format 'code={{.State.ExitCode}}' s-script
```

Observez 1 {entrypoint.sh} /bin/sh /entrypoint.sh et 7 java -jar /app/api.jar, puis les 10 secondes et le code 137 — **même sur Alpine.**

Corrigez en ajoutant `exec` :

```
printf '#!/bin/sh\neco "preparation..."nexec java -jar /app/api.jar\n' > entrypoint.sh
docker build -q -f D-script -t api-labo:script2 .
docker rm -f s-script && docker run -d --name s-script api-labo:script2 ; sleep 3
docker exec s-script ps -o pid,args | head -3
time docker stop s-script ; docker inspect --format 'code={{.State.ExitCode}}' s-script
docker rm s-script
```

Observez qu'il n'y a plus de shell, l'arrêt est instantané et le code est 143 .

Étape 8 — ARG n'est pas un coffre-fort

```
printf 'FROM alpine\nARG DB_PASSWORD=vide\nRUN echo "build avec $DB_PASSWORD" > /trace.txt\nCMD ["cat", "/trace.txt"]\n' > D-arg
docker build -q -f D-arg --build-arg DB_PASSWORD='Secr3t!' -t api-labo:arg .
docker image inspect --format '{{json .Config.Env}}' api-labo:arg
docker history --no-trunc api-labo:arg --format '{{.CreatedBy}}' | head -3
```

Observez que `Config.Env` ne contient pas le mot de passe (c'est vrai, `ARG` ne devient pas `ENV`)... mais que `docker history` affiche `ARG DB_PASSWORD=Secr3t!` et `RUN |1 DB_PASSWORD=Secr3t! ...`.

Explication. Le secret est dans l'image, lisible par quiconque la possède. La bonne méthode viendra au labo 08.

Étape 9 — USER et son placement

```
printf 'FROM eclipse-temurin:21-jre-alpine\nUSER 1000:1000\nWORKDIR /app\nRUN mkdir /data\nCOPY api.jar /app/api.jar\nENTRYPOINT ["java", "-jar", "/app/api.jar"]\n' > D-user-tot
docker build --progress=plain -f D-user-tot -t api-labo:user-tot . 2>&1 | grep -iE 'permission|ERROR'
```

Observez l'échec : `mkdir: cannot create directory '/data': Permission denied`, puis `ERROR: process "/bin/sh -c mkdir /data" did not complete successfully: exit code: 1`.

Placez `USER` en dernier :

```
printf 'FROM eclipse-temurin:21-jre-alpine\nWORKDIR /app\nRUN mkdir /data && chown 1000:1000 /data\nCOPY --chown=1000:1000 api.jar /app/api.jar\nUSER 1000:1000\nENTRYPOINT ["java", "-jar", "/app/api.jar"]\n' > D-user-ok
docker build -q -f D-user-ok -t api-labo:user-ok .
docker run --rm --entrypoint id api-labo:user-ok
```

Observez `uid=1000 gid=1000 groups=1000` : l'application ne tourne plus en root.

Explication. `USER` s'applique à tout ce qui suit, build **et** exécution. On le place donc juste avant `ENTRYPOINT`, après avoir préparé les fichiers et ajusté leurs propriétaires.

Nettoyage

```
docker rm -f api 2>/dev/null
docker rmi api-labo:1.0 api-labo:2.0 api-labo:2.1 api-labo:3.0 api-labo:3.1 \
    api-labo:cmd api-labo:shell-debian api-labo:shell-alpine \
    api-labo:script api-labo:script2 api-labo:arg api-labo:user-ok 2>/dev/null
docker images --format '{{.Repository}}:{{.Tag}}' | grep api-labo
rm -rf ~/labo-docker/04/node_modules ~/labo-docker/04/./commun
```

Gardez `~/labo-docker/04/api.jar` et `Api.java` : les labos 05 à 09 les réutilisent. Conservez aussi les images `eclipse-temurin:21-jre-alpine` et `eclipse-temurin:21-jdk`.

Ce que vous devez pouvoir affirmer maintenant

- Le contexte détermine ce qui est copiable et ce qui est transféré ; `.dockerignore` est indispensable.
- Une instruction invalidée invalide toutes les suivantes : l'ordre décide du temps de build.
- `EXPOSE` ne publie rien.
- `ENTRYPOINT` fixe le programme, `CMD` donne des arguments remplaçables, `--entrypoint` permet de déboguer.
- La forme *shell* peut casser l'arrêt propre — et son comportement dépend de l'image de base. Vous l'avez mesuré : 0,2 s / code 143 contre 10 s / code 137.
- Un script d'entrée doit finir par `exec`.
- Un `--build-arg` est visible dans `docker history`.
- `USER` se place juste avant `ENTRYPOINT`.